Ders 8 Çalışma Özeti

Ders 8 Çalışma Özeti

Genel Konular

- Bölüm 5'e giriş: bağlam farkında sinirsel dil modelleri
  - Bu derste Bölüm 5'e ("Bağlam Farkında Sinirsel Dil Modelleri", ekrandan, 00:10:00) başlandı. Önce bağlam tabanlı modellerin yapı taşları (dağıtık temsil, RNN, kodlayıcı-kod çözücü, dikkat, Transformer) anlatıldı, sonra bağlam tabanlı dil modelleri, kendi kendine öğrenme görevleri ve alt kelime (subword) temsili ele alındı.
- Dağıtık temsil (distributed representation) (ekrandan, 00:11:30)
  - Resimlerdeki insanları kadın/erkek, gözlüklü/gözlüksüz, uzun/kısa diye ayırmak istersek 8 sınıf tanımlayabiliriz; bu durumda her sınıftan örneğe ihtiyaç olur.
  - Bunun yerine 3 alt özelliği öğrenen 3 ikili (binary) sınıflandırıcı tanımlanırsa, eğitim kümesinde "uzun, kadın, gözlüksüz" bir örnek olmasa bile bu kombinasyon tanınabilir.
  - d adet alt özellikle 2^d farklı kombinasyon ifade edilebilir. N sınıf, birbirini dışlamayan (non mutually exclusive) özellikler taşıyorsa $\log_2 N$ alt özellikle ifade edilebilir.
  - Sayısal örnek (ekrandan, 00:15:10): 1024 sınıf ve $d \times d$ boyutlu resimler için klasik yaklaşımda $(d \cdot d) \cdot 1024$ parametre gerekir. 10 alt özellik kullanılırsa ağa bir katman eklenir ve $(d \cdot d) \cdot 10 + 10 \cdot 1024$ parametre öğrenilir. $d = 128$ için distributed yaklaşım yaklaşık 180 bin parametredir (tam değer 174.080). Katman sayısı arttırılarak öğrenilecek parametre sayısı azaltılmış olur.
  - Ders sonrası not: slayt ve hoca klasik yaklaşım için "yaklaşık 4 milyon (2^{24})" dedi; doğrusu $128 \cdot 128 \cdot 1024 = 2^{24} = 16.777.216$, yani yaklaşık 16,8 milyondur.
- Alt özellikler nasıl belirlenir? (ekrandan, 00:20:00)
  - Elle tanımlanmış kurallarla belirlemek zordur; uçtan uca öğrenme (end-to-end learning) ile öğrenilir. Bir MLP'nin her gizli katmanındaki her nöron, bir CNN'deki her filtre bir alt özellik öğrenir.
  - Varsayım: tüm resimler ortak küçük parçalardan oluşur. Yüz, araba, fil ve sandalye için eğitilmiş ağların ilk katmanlarındaki özellikler (yatay, dikey çizgi gibi) birbirine çok benzer; üst katmanlarda probleme özgü hale gelir.
  - Bu, öğrenme transferinin (transfer learning) neden mümkün olduğunu açıklar: alt seviye temsiller büyük veriyle bir kez öğrenilir, yeni görevde yalnız probleme özgü üst katmanlar değiştirilir.
- Kapsül ağları (Capsule Networks) (ekrandan, 00:25:30)
  - CNN'de küçük parçaların resmin herhangi bir yerinde var olması yeterlidir; pooling katmanı nerede olduğunu önemsizleştirir, sadece olup olmadığını söyler.
  - Oysa parçaların birbirine göre konumları da önemlidir: yüzde ağızla gözün yeri değiştirilirse hâlâ yüz müdür? Hinton'un önerdiği kapsül ağları bu göreceli konumları da işe katar. Ayrıntısına girilmedi.
- Yinelemeli sinir ağları (RNN) (ekrandan, 00:30:00)
  - Girişin sekans olduğu ve sıranın önemli olduğu uygulamalar içindir: bir cümlenin kelimeleri, bir filmin kareleri, bir konuşmanın fonemleri.
  - h_t: t. çıkış, x_t: t. giriş. h_t, x_t ve h_{t-1}'e bağlıdır; aslında $x_0 \dots x_t$'nin hepsine bağlıdır.
  - Açılım (ekrandan, 00:40:00): $h_3 = f(x_3, h_2) = f(x_3, f(x_2, h_1)) = f(x_3, f(x_2, f(x_1, h_0)))$.
  - Sabit uzunluklu klasik ağlar da geçmişi kullanabilir: giriş olarak $(x_t, x_{t-1}, \dots, x_{t-k})$ birleştirilerek verilir. Slayttaki örnek (ekrandan, 00:37:40): giriş 5 kelime, çıkış sonraki kelime; "Ali bugün okula giderken yolda → arkadaşı", "bugün okula giderken yolda arkadaşı → Hilmiyi", "okula giderken yolda arkadaşı Hilmiyi → gördü". Trans-

former'lar da sabit uzunlukla çalışır. Fark: RNN'in uzunluk sınırı teoride sonsuzdur, sabit yapılarda bir sınır vardır.

- RNN uygulama türleri (ekrandan, 00:42:00)
  - one to one: klasik sinir ağı. one to many: resim girer, onu anlatan cümle çıkar. many to one: tweet girer, duygusu (sentiment) çıkar. many to many: cümle girer, başka bir dildeki karşılığı çıkar.
- RNN denklemi ve öğrenilen ağırlıklar (ekrandan, 00:45:00)
  - $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$. 3 saklı nöronlu bir RNN 3 boyutlu bir durum (state) tutar; durumlar o ana kadar gelen girişlerin temsilidir.
  - W_{xh} girişten saklı katmana, W_{hh} önceki saklı durumdan şimdikine geçişi öğrenir. Çıkış üretmek için ayrıca W_{hy} öğrenilir.
  - Hoca durumu sonlu durum makinelerine benzetti: farkı, durum sayısının sabit olmaması ve durumun ayrık değil, sürekli bir uzayda bir vektör olmasıdır. W_{hh} girdiden bağımsız durum geçişini, W_{xh} girdiye bağlı geçişi modeller.
- RNN ile bir sonraki harfi tahmin (ekrandan, 00:50:00)
  - Sözlük 4 karakter (h, e, l, o), giriş ve çıkış 4 boyutlu one-hot, 3 saklı nöron. "hell" girildiğinde hedef karakterler "ello"dur.
  - Kelime düzeyinde one-hot kullanmak sözlük boyutunu çok büyütür; karakter ile kelimenin iyi yanlarını birleştiren çözüm alt kelimelerdir.
- RNN türevleri
  - LSTM, BiLSTM, GRU gibi yapıların hepsi RNN ailesindendir; RNN'lerin yapamadığı bazı şeyleri yapan alt türlerdir.
  - LSTM ve GRU, geçmişin ne kadarının hatırlanıp ne kadarının unutulacağını (bazı x_t'lerin h'yi etkilememesini) dinamik olarak öğrenen daha uyarlanabilir yapılardır.
  - GRU (ekrandan, 01:17:40): $z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$, $r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$, $\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$, $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$. LSTM'e göre daha basittir; 3 W matrisi (W_z, W_r, W) öğrenilir.
- RNN'lerle dil çevirisi: kodlayıcı-kod çözücü (ekrandan, 01:05:30)
  - Çeviride kaynak cümlenin tamamı görülmeden hedef cümleye başlamak anlamlı değildir, çünkü dillerin grameri farklıdır.
  - Kodlayıcı: $h_t = \varphi(h_{t-1}, x_t) = f(W^{hh}h_{t-1} + W^{hx}x_t)$. Kod çözücü: $h_t = \varphi(h_{t-1}) = f(W^{hh}h_{t-1})$, $y_t = \text{softmax}(W^S h_t)$.
  - Kodlayıcının son durumu tüm cümlenin temsilidir (örneğin 300 boyutlu). W^S, word2vec'in çıkış tarafına benzer biçimde 300 boyutlu vektörü sözlük boyutuna (örneğin 32 bin) softmax ile dönüştürür.
- Dil çevirisinde bazı iyileştirmeler (ekrandan, 01:10:00)
  - Tüm cümle tek bir duruma (C) indirgenmiştir; bu zor bir görevdir.
  - Kodlayıcıya cümleyi tersten vermek: kodlanacak ve çözülecek bilgiler arasındaki mesafe azalır; cümlenin başı daha kolay üretilir. Küçük bir iyileşme sağlar.
  - Kod çözücü: $h_t = \varphi(h_{t-1}, C, y_{t-1})$; her biri için ayrı bir W vardır. Çıkış: $y_t = \text{softmax}(W^S[h_t; C])$.
  - Teoride bu eklemeler gereksizdir (C ve y_{t-1} zaten h'den türetilir), ama pratikte işi kolaylaştırdıkları görülmüştür.
- Dikkat mekanizması (attention) (ekrandan, 01:20:00)
  - Transformer'lardan önce de vardır. Kodlayıcının tüm durumları etkindir, ama ne ölçüde etkin (α) olacakları öğrenilir.
  - $\alpha^{i,j}$: i. çıkışa j. girişin ne kadar etkisi olacağı. $\sum_t \alpha^{1,t} = 1$, $h^t = (h^{\rightarrow t}, h^{\leftarrow t})$ (çift yönlü), $C_1 = \sum_t \alpha^{1,t} h^t$.
  - Çinceden İngilizceye çeviren animasyonda çizgilerin kalınlığı, girişteki kelimenin çıkıştaki kelimeye etkisini gösterir. Parametre sayısını arttırır ama performansa katkısı çok büyüktür.
- Transformer'lar (ekrandan, 01:23:00)
  - "Attention is all you need" makalesi; tamamen bir makine çevirisi makalesidir. RNN yoktur, dolayısıyla paralelleştirilebilir. Sıra bilgisi konum kodlamasıyla (position encoding) verilir.
  - Klasik attention giriş ve çıkış dizisi arasındaki bağlılıkları bulur. Transformer'lar girişin ve çıkışın kendi içindeki bağımlılıklarını da işe katar: öz-dikkat (self-attention). Multihead attention aynı şeyin birden fazla temsilini oluşturur (8 başlık, 8 temsil).

  - Öz-dikkat örneği (ekrandan, 01:30:00): “Garajımda 300’le gidebilen bir jaguar var.” ve “Ormanda beni jaguar kovaladı.” Jaguar’ın anlamını çevresindeki kelimeler (garaj, 300’le gidebilmek; orman, kovalamak) belirler.
- Bağlam tabanlı dil modelleri (ekrandan, 01:40:00)
  - word2vec’te her kelimenin bağlamdan bağımsız tek bir vektörü vardır; “jaguar”ın vektörü hem arabaya hem hayvana yakındır. Bağlam tabanlı modellerde vektör cümleye göre değişir; bu yüzden kelime vektörleri bir yerde saklanamaz, model saklanır ve cümle modele verilir.
  - Bağlamın etkisi (ekrandan, 01:34:20): aynı şekil “11 12 *” yatay bağlamında 13, ”A* C” dikey bağlamında B olarak görülür.
  - RNN tabanlı yaklaşımlar: TagLM (sonraki kelimeyi tahmin için önceden eğitilmiş modelin başka bir görevde ilk kullanımı), ELMo (çok sayıda görevde performans artışı).
  - Transformer tabanlı yaklaşımlar: GPT (sonraki kelimeyi tahmin), BERT (maskelenmiş kelimeyi tahmin ve sonraki cümle mi ikili sınıflandırma; pozitif: ardışık iki cümle, negatif: rastgele iki cümle), RoBERTa (yalnız maskelenmiş kelime), ALBERT (maskelenmiş kelime ve sonraki cümle mi; negatif örnek: pozitifteki iki cümlenin yeri değiştirilmiş), ELECTRA (değiştirilmiş kelimeleri tahmin), BART (çeşitli görevler).
- Kendi kendine öğrenme görevleri (self-supervised) (ekrandan, 01:44:40)
  - Büyük miktarda eğitim verisi üretmek için kurulan yan görevlerdir; insan etiketlemesine/emeğine ihtiyaç yoktur. Sonraki kelime(ler)i, cümleyi ya da film karesini tahmin etmek; gürültü ekleme: maskelenmiş kelime(ler)i tahmin, sırası bozulmuş cümleyi düzeltme, harfleri bozulmuş kelimeyi düzeltme, resmin silinmiş kısmını tahmin.
  - BART görevleri: Token Masking (A_C._E.), Token Deletion (A.C.E.), Text Infilling (A_.D_E.), Sentence Permutation (DE.ABC.), Document Rotation (C.DE.AB); hepsinde hedef özgün cümledir (ABC.DE.).
  - RNN’ler kelimeleri teker teker aldığı için kolay paralelleştirilemez; Transformer’lar tüm diziyi tek seferde aldığı için çok daha hızlıdır. Kendi kendine öğrenme ile Transformer’ın hızı birleşince veri kıtlığı problemi aşıldı.
- Alt kelimeler (subword) ve Byte Pair Encoding (ekrandan, 02:08:40 ve 02:10:00)
  - Kelime düzeyi: sözlük çok büyür (bir derlemde 1 milyon farklı kelime olabilir). Karakter düzeyi: her harften anlam çıkarmak optimal değildir. Orta yol alt kelimelerdir: “Gele #ceğ im”.
  - Alt kelimeler sözlük boyutunu sınırlar ve sözlük dışı kelime (out of vocabulary, OOV) problemini çözer: en kötü durumda kelime harflerden kurulur. Türkçe gibi morfolojik olarak zengin dillerde bu çok önemlidir (“geleceğim” eğitimde olup “gelmeyeceğim” olmayabilir).
  - BPE bir veri sıkıştırma yönteminden gelir: en sık geçen bitişik ikili (bigram) yeni bir sembolle değiştirilir, işlem tekrarlanır. Slayttaki örnek: aaabdaaabac → ZabdZabac (Z=aa) → ZYdZYac (Y=ab) → XdXac (X=ZY).
  - “##” ile başlayan parça bir kelimenin devamıdır (ek). BERT sözlükleri 32K, 64K gibi boyutlardadır; hoca 32K’dan 64K’ya çıkmanın performansta çok büyük fark yaratmadığını söyledi.
  - BPE ilk popülerliğini makine çevirisinde nadir kelimelerin temsilinde kazandı. WordPiece ve SentencePiece bunun gerçeklemeleridir.

Hocanın Özellikle Vurguladığı Kısımlar

- Dağıtık temsil sinir ağlarının en güçlü yanıdır
  - Sınıfları alt özelliklerin kombinasyonu olarak düşünmek hem parametre sayısını düşürür hem de eğitimde hiç görülmemiş kombinasyonların tanınmasını sağlar.
- LSTM, GRU ve BiLSTM ayrı şeyler değil, RNN’dir
  - Hoca bunları gördüğünüzde RNN olduklarını bilmenizi istedi; temel denklemden çok büyük farkları yoktur, yalnız daha uyarlanabilirdirler.
- Asıl çığır açan öz-dikkattir
  - Giriş-çıkış dikkatinden dizinin kendi içindeki dikkate geçiş kritik noktadır. Multihead ve key-value gibi ayrıntılar pratik ayrıntılardır; ileride değişebilir.
- Kendi kendine öğrenme ölçeklenebilirlik problemini çözdü
  - Derin modeller çok büyük veriyle iyi çalışır, büyük veriyi insanlara etiketletmek mümkün değildir. İnsan etiketi gerektirmeyen görevlerle bütün metinler eğitim kümesine dönüşür.

- Makine çevirisi doğal dil işlemenin lokomotif problemidir
  - Kodlayıcı-kod çözücü, dikkat, Transformer ve BPE bu problemle ortaya çıktı ya da popülerleşti.
- Alan çok hızlı değişiyor
  - 2020'deki genel resim 2023'te epey değişti; sonraki token tahmini (GPT tarafı) standart yaklaşım haline geldi.

Kısa Tekrar Notları

- d alt özellik → 2^d kombinasyon; N sınıf → $\log_2 N$ alt özellik.
- $d = 128$, 1024 sınıf: klasik 2^{24} (yaklaşık 16,8 milyon), dağıtık 174.080 (yaklaşık 180 bin) parametre.
- CNN'lerin alt katmanları farklı görevlerde benzer: transfer öğrenmenin temeli.
- Kapsül ağları: parçaların birbirine göre konumu.
- RNN: $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$; çıkış için W_{hy}.
- RNN türleri: one to one, one to many, many to one, many to many.
- Kodlayıcı-kod çözücü: tüm cümle tek vektör C; iyileştirmeler: ters giriş, C ve y_{t-1}'i kod çözücüye vermek.
- Dikkat: $C_i = \sum_t \alpha^{i,t} h^t$, $\sum_t \alpha^{i,t} = 1$.
- Transformer: RNN yok, paralel, konum kodlaması, öz-dikkat, multihead.
- word2vec: kelime başına tek vektör; bağlam tabanlı: cümleye göre değişen vektör.
- GPT: sonraki token; BERT: maske + sonraki cümle; RoBERTa: yalnız maske; ALBERT: maske + cümle sırası; ELECTRA: değiştirilmiş kelime; BART: gürültü giderme.
- BPE: en sık bigram'ı yeni sembolle değiştir, tekrarla; OOV problemi çözülür.

Detaylı Açıklamalar

- Dağıtık temsilin parametre kazancı
  - Klasik yaklaşımda her sınıf için bir prototip öğrenilir: $d \times d$ boyutlu girişten 1024 çıkışa doğrudan bağlantı $(d \cdot d) \cdot 1024$ ağırlık ister. Araya 10 nöronlu bir katman eklenince ağ önce 10 alt özelliği, sonra bu özelliklerin kombinasyonlarını sınıf etiketlerine eşlemeyi öğrenir: $(d \cdot d) \cdot 10 + 10 \cdot 1024$. $d = 128$ için $16384 \cdot 10 + 10240 = 174.080$ parametre elde edilir. Katman sayısı arttığı halde parametre sayısı azalmıştır.
- RNN'in geçmişi nasıl taşıdığı
  - $h_t = f(x_t, h_{t-1})$ yazılıp h_{t-1} yerine kendi tanımı konunca $h_3 = f(x_3, f(x_2, f(x_1, h_0)))$ elde edilir; yani herhangi bir andaki çıkış o ana kadarki bütün girişlere bağlıdır. Sabit pencereli bir ağda ise 5 kelimelik pencere seçilirse altıncı çıkış birinci kelimeden etkilenemez. Pratikte pencere yeterince geniş tutulursa bu çok sorun olmaz; ama teoride RNN'in sınırı sonsuzdur.
- Kodlayıcı-kod çözücüde eğitilen parçalar
  - Kodlayıcıda W^{hx} (girişten durum) ve W^{hh} (durumdan durum), kod çözücüde ayrı bir W^{hh} ve çıkış için W^S öğrenilir. Kodlayıcı ve kod çözücüdeki matrislerin aynı olması gerekmez.
- Dikkatin sezgisi
  - Tek bir C vektörüne tüm cümleyi sıkıştırmak yerine, her çıkış kelimesi için kodlayıcının bütün durumlarının ağırlıklı toplamı alınır. Ağırlıklar (α) öğrenilir ve her çıkış için toplamları 1'dir. Böylece örneğin çıkışın ilk kelimesini girişin ilk iki kelimesi, ikinci kelimesini sonraki iki kelimesi en çok etkileyebilir.
- Bağlam tabanlı modellerin bedeli
  - Kelime vektörü bağlama göre değiştiği için word2vec'teki gibi "kelime → vektör" tablosu saklanamaz. Elde bir model tutulur; cümle modele verilir ve kelimenin o cümledeki vektörü hesaplanır.
- Byte Pair Encoding adımları
  - aaabdaaabac dizisinde en sık bitişik ikili "aa"dır (örtüşmeli sayımla 4 kez; değiştirmede 2 yer). Z=aa ile ZabdZabac olur. Ardından "ab" (2 kez) Y yapılır: ZYdZYac. Sonra "ZY" (2 kez) X yapılır: XdXac. Tekrar eden ikili kalmayınca durulur. Sözlük X, Y, Z ve bunların karşılıklarından oluşur; X = aaab'dir. Doğal dilde sık geçen ekler ve kökler bu yolla sözlüğe girer.

  - Ders sonrası not: ikinci adımda “Za” ve “ab” ikilileri eşit sıklıktadır (ikisi de 2 kez); slayt “ab”yi seçmiştir (python ile sayıldı).
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.